

Credo: Reusable Declarative Primitives for Agentic Workflows

Duo Lu
Brown University
Providence, RI, USA
duo_lu@brown.edu

Andrew Crotty
Northwestern University
Evanston, IL, USA
andrew.crotty@northwestern.edu

Uğur Çetintemel
Brown University
Providence, RI, USA
ugur_cetintemel@brown.edu

Abstract

An LLM application depends on both a model and a *harness*: the program that determines what each call sees, how many calls to make, and which answers to trust. Coding agents can now discover strong harnesses by searching over candidate programs, but the resulting artifact is an opaque block of imperative code whose logical steps, runtime signals, physical execution decisions, and prompt strategies remain implicit and task-specific, forcing subsequent tasks to start the harness search process from scratch.

The potential for reuse, however, is substantial. A searched harness encodes significant knowledge, such as the logical steps that work, the signals that matter, the physical operator decisions that adapt execution, and the prompt strategies that are effective, yet this knowledge is buried in imperative code with no inspectable or reusable structure, nor does it carry any provenance or metadata.

CREDO addresses this problem by recovering a structured declarative description of a searched harness, tagging each extracted primitive with relevant metadata, and cataloguing all of it with provenance. A compiler can then bind stored primitives to generate harnesses for new tasks without having to start the search over from scratch. This paper provides preliminary results demonstrating the potential of our approach and lays out a related research agenda that the database community is well-positioned to tackle, including cost-based compilation over declarative catalogs and catalog maintenance under model and workload drift.

1 Introduction

The behavior of an LLM application depends heavily on its *harness*, which determines the sequence of model and tool calls, the information supplied to each call, and the conditions under which an answer is accepted. For example, a harness may begin with a small model, estimate confidence in the resulting answer, and invoke a larger model only when that estimate falls below a threshold. Thus, both quality and cost depend on program-level choices that can extend well beyond the model itself.

Recently, agentic harness search has become an effective way to make these choices. A search agent proposes candidate harnesses and executes them against a benchmark, using the observed accuracy and cost to guide subsequent iterations [16], following the evolutionary pattern coding agents apply to programs more generally [22]. The final program encodes one successful combination of decisions as a single monolithic output, such that the logical sequence of operations, intermediate signals, conditional implementation choices, and prompt strategies all appear mixed together, along with constants and conventions specific to the benchmark. As a result, knowledge that might be generalizable to other tasks is lost among the mess.

Database systems have faced a similar problem in the past. For decades, user-defined functions (UDFs) remained as black boxes

that could not be optimized or reasoned about, but work on UDF introspection and recovery of declarative expressions from procedural code has exposed opportunities that were previously hidden from the optimizer [8, 25]. We therefore argue for a similar approach for LLM harnesses, such that their internals can be decomposed into reusable declarative primitives.

CREDO applies these lessons to LLM harnesses by attempting to recover a typed declarative description as primitives of four kinds: (1) *skeleton steps* forming the logical plan; (2) *beliefs* as derived attributes that inform physical operator selection; (3) *policies* as the rules that select physical implementations using belief values; and (4) *prompt templates* as the parametric natural language specifications discovered at each step. Every primitive is stored with provenance and relevant metadata, allowing a compiler to synthesize harnesses for new tasks from the catalog by reusing extracted primitives. The key insight is that declarative descriptions of agentic execution are portable and durable, meaning they can be recovered once during harness search and then compiled into new harnesses across new domains, accumulating in a catalog that grows more valuable with every new task. Our preliminary results demonstrate the promise of our proposed approach, which achieves more than an order of magnitude in cost savings over agentic harness search. We also outline our vision for a broader research agenda this line of work opens up for the database community.

2 The CREDO Framework

The input to CREDO is a searched harness.py, and its output is a declarative description from which a harness can be reconstructed or adapted. The description must retain enough operational detail to reproduce behavior, while separating the method discovered by search from decisions tied to the source dataset. Fig. 1 illustrates the process using a harness for a finance workload that is recovered into typed primitives, stored in a catalog, and subsequently recompiled into a harness for a legal workload.

2.1 Structured Declarative Description

The declarative description encodes a harness as structured primitives expressed in natural language (NL). Beliefs and policies keep the control-plane semantics of our earlier CREDO system [20], where a developer authored them against their own pipeline, whereas here they are automatically extracted from a searched program, along with the skeleton and templates. CREDO extracts the following four types of reusable primitives.

Skeleton. The skeleton is the declarative logical plan, which is a directed acyclic graph (DAG) of named operator steps and the dependencies between them, recording what the harness computes and in what order. The blue boxes in Fig. 1 are the skeleton the abstractor recovers from the finance harness: `parse_query`, `retrieve`, `rerank`, `generate`, and `self_check`. It is analogous to a query

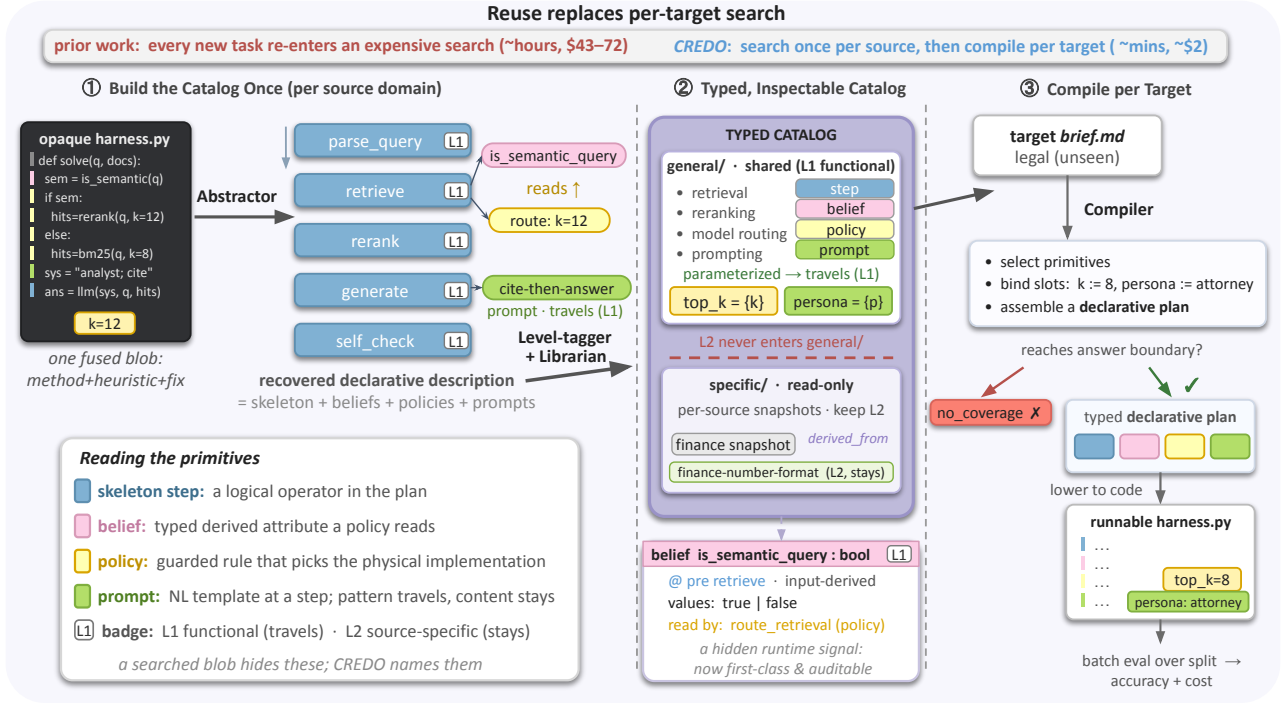


Figure 1: CREDO recovers a declarative description from a searched harness once (abstractor), files its transferable primitives in a shared catalog (level-tagger, librarian), and later compiles a harness for an unseen target out of that catalog (compiler) rather than searching again.

plan that describes the shape of the computation, separating logical intent (what each step does) from physical implementation (how it runs). For now, we do not assume any formal equivalence rules for reordering or replacing steps, though this presents an interesting research opportunity (see §4).

Beliefs. Beliefs are the derived attributes the harness computes at declared plan points and reads to make physical operator decisions. Each has a name, a type from a closed value set {bool, enum, int, float, str}, a point in the skeleton where it becomes available, and the policies that consume it. Beliefs fall into three categories: (1) *input-derived* attributes computed from the raw input before any operator runs (e.g., `is_semantic_query : bool`); (2) *intermediate-result* attributes computed from operator outputs and carried forward (e.g., `extraction_confidence : float`); and (3) *output-quality* attributes computed over final or near-final output to govern acceptance or escalation (e.g., `schema_match : bool`). In the finance harness, `is_semantic_query` was a local, implicit branch condition that CREDO can promote to a declared, typed, named plan primitive so it can be logged, compared across runs, and traced to any associated policies that rely on it.

Policies. Policies are akin to physical operator selection rules in a database system. Each policy evaluates a condition over one or more beliefs and, when it holds, selects the physical implementation of a logical step (e.g., model, parameters, execution strategy). The yellow primitive in Fig. 1 is an example policy. At `retrieve`, when `is_semantic_query` holds, select dense retrieval with `k=12`;

otherwise, select keyword search with `k=8`. Separating what a step is for from how it runs is the same as the logical/physical plan split in database query planning, where an operator’s declarative behavior is decoupled from its actual implementation.

Prompt templates. Prompt templates are the parametric NL specifications the search discovered at each skeleton step (i.e., the physical operator implementations themselves). Each template captures the reasoning strategy, instruction pattern, output format, or persona the search found effective, expressed as NL templates with bindable slots. The two green primitives in Fig. 1 are templates, including a generic `cite-then-answer` pattern that travels to legal tasks and a finance-specific number-format nudge that stays local.

2.2 Building and Serving the Catalog

CREDO processes the harness in two steps: (1) an offline catalog construction phase, and (2) an online compilation phase. During catalog construction, the abstractor extracts the primitives, the level-tagger determines their transfer scope, and the librarian records the results. Search and extraction are therefore performed for a source harness once, while later targets can reuse the resulting primitives through a substantially cheaper compilation step.

Building the catalog. Every primitive carries a transfer scope tag. A primitive is *functional* (L1) when its pattern recurs across a task family and can be lifted into the shared catalog; it is *source-specific* (L2) when its pattern encodes one dataset’s conventions and cannot be lifted out. Scope is judged on the pattern, not the

Table 1: Evaluation datasets and the benchmarks each split draws from. The legal datasets are all taken from the LegalBench-RAG bundle [24].

Dataset	Domain	Search / In-Distribution Sources	Cross-Distribution Sources
Finance	Retrieval	DocFinQA [27] + FinQA [4]	FinanceBench [13] + TAT-DQA [47] + TAT-HQA [17]
Legal	Retrieval	CUAD-common [12] + MAUD [36]	ContractNLI [15] + PrivacyQA [26] + CUAD-rare [12]
Math	Reasoning	OlympiadBench [11] + Omni-MATH [9]	IMO-AnswerBench [21] + IMO-ProofBench [21] + ArXivMath [6]
Polyreason	Reasoning	GPQA-Diamond chem+bio [28]	MMLU-Pro law+phil [37] + ZebraLogicBench [18] + MuSR [31]

surface text or constants a primitive carries. CREDO first lifts source-specific constants into bindable slots (e.g., a retrieval step that hard-codes `top_k=12` instead exposes `top_k={k}`), but parameterizing a constant does not by itself make a primitive transferable; rather, only a pattern that survives the lift does. The tag is a binary gate, such that functional primitives enter the shared catalog and source-specific ones stay with their source.

Once a declarative description is recovered and its primitives are scope-tagged, the librarian files them in a catalog with two tiers. The `general/` tier is shared, holding the parameterized functional (L1) primitives indexed by function (e.g., retrieval, reasoning) rather than by subject, with each entry recording the provenance of which harness(es) it came from. The `specific/` tier is the per-harness record, and it keeps the entire description as source-specific primitives, such that promoting part of a description to `general/` never discards the rest. As the names suggest, the `general/` tier is primarily intended for reuse, whereas the `specific/` tier is only relevant for individual harnesses and primarily maintained for provenance purposes. Thus, all of the resulting primitives remain both inspectable and traceable throughout.

Serving a target. The compiler selects primitives to connect the target’s input to an answer, binds their open slots to target-specific values, and assembles a runnable `harness.py`. If no combination reaches a satisfactory threshold, CREDO simply falls back to the harness search process. Importantly, the catalog retains only primitive descriptions, not any implementation details.

These two phases also define how we evaluate CREDO (§3.2 and §3.3). Recovering a declarative description and compiling it straight back, then running the original and the compiled harness on the same held-out split, is an empirical fidelity check, such that matching both accuracy and cost is evidence that the compiled harness faithfully captures the original’s execution method. Compiling a harness for a new target from primitives on other domains tests whether reuse can yield a working harness without a new search. We discuss these further in the next section.

3 Preliminary Results

This section describes our preliminary results. We organize our experiments around the following research questions:

- **Catalog Construction (§3.1).** Can CREDO recover useful primitives from a searched harness?
- **Declarative Fidelity (§3.2).** Can a recovered declarative description preserve the behavior of the harness it describes?
- **Primitive Reuse (§3.3).** Can catalogued primitives be compiled into a working harness for an unseen target without re-searching?

System and Models. All experiments run on a single dual-socket server with 2× AMD EPYC 9554 processors and 755 GiB of RAM, hosting 4× NVIDIA L40S GPUs (48 GB each, 192 GB aggregate). The open-weight deployment models (the small / large pair, i.e., `gemma3-12b / gemma4-31b`) are served locally with vLLM (v0.20.1) behind OpenAI-compatible endpoints across the 4× L40S GPUs. The harness discovery agent (proposer) and CREDO’s abstractor and compiler run on a frontier model (`claude-opus-4-7`), while the accuracy judge runs on `gpt-5-mini`. These are API calls billed at list rates, and their latency and dollar cost are reported separately from the local open-weight inference.

Datasets. We use four datasets spanning two functional domains: *retrieval* (finance, legal) and *reasoning* (math, polyreason). Every dataset uses the same split: (1) search, which the harness discovery agent iterates against (100 queries); (2) in-distribution, which is held out from the same source mix as search and used to select the best harness programs from the Pareto frontier (100 queries); and (3) cross-distribution, which is aligned with search but structurally distinct in document register, question template, or content domain (300 queries). Table 1 summarizes the datasets. Note that *domain* refers to the workload we construct, not the benchmark’s native form. For the retrieval domains, we discard each question’s gold evidence attachment and point the harness at a pool directory of source documents, so locating the evidence is part of the task.

Metrics. We report two measures per harness: task *accuracy* (`gpt-5-mini` judge, per-dataset grading rules) and harness *cost*, the per-question inference cost (¢/q). Accuracy alone is misleading (e.g., a harness can often raise it with more compute), so we report both and compare them together rather than combining into one score [39]; related work likewise treats cost reduction as constrained by an accuracy guarantee rather than traded against it freely [43]. Results vary from run to run, so we use as many seeds and replicates as our budget allows.

Harness Search and Discovery. We discover harness programs with Meta-Harness [16], a state-of-the-art coding-agent search that browses all prior candidates’ source, scores, and traces. Each search runs 10 iterations, and we select programs on the Pareto frontier of the accuracy and cost trade-off (in-distribution split)¹. Since CREDO’s recovery is agnostic to how a harness was produced, we fix the searcher rather than compare searchers. Every agent also runs inside a sandbox that prevents evaluation leakage.

¹<https://github.com/stanford-iris-lab/meta-harness>

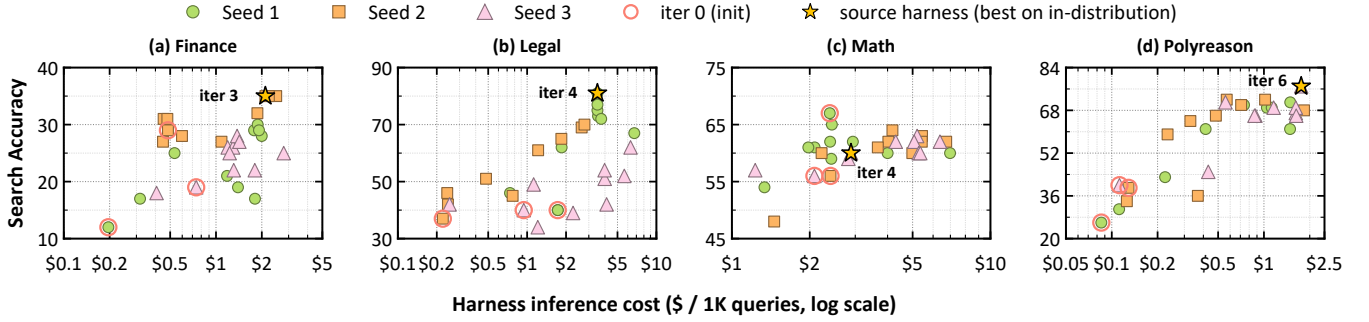


Figure 2: Meta-Harness search trajectories, one plot per domain; each marker is one search iteration. Stars mark each domain’s best program on the *in-distribution* split, which need not be the topmost point on the search-split axis.

Table 2: Round-trip fidelity and cost, per domain over 3 seeds $\times$ 3 compile replicates (= 9 runs); $\Delta = \text{RT} - \text{orig}$. *Compile* is one recover $\rightarrow$ compile call; *Meta-Harness search* splits the full search into its proposer and evaluation halves.

Domain	Round-trip accuracy			Harness $\text{\$/q}$			Compile		Meta-Harness search			
	Orig	RT	mean Δ	Orig	RT	mean Δ	\$	s	Prop \$	Eval \$	Prop wall	Total wall
Finance	0.293	0.304	+0.011	0.171	0.170	−0.000	2.79	134	67	1.4	41 m	2.8 h
Legal	0.673	0.667	−0.007	0.420	0.413	−0.007	2.80	173	72	2.6	41 m	4.4 h
Math	0.618	0.608	−0.010	0.363	0.376	+0.013	2.25	94	44	4.5	30 m	6.8 h
Polyreason	0.710	0.717	+0.007	0.130	0.119	−0.011	2.42	98	43	0.8	30 m	9.2 h

3.1 Catalog Construction

Since CREDO takes a searched harness as input, we first run Meta-Harness on each domain to produce a strong harness. Meta-Harness begins its search from a *brief.md* that specifies the task environment and answer interface without supplying a solution strategy. The search agent can invoke both the small and large models, which permits candidate programs such as small-to-large cascades, and a subagent retains the candidates on the accuracy-cost Pareto frontier. As shown in Fig. 2, the three searches for each content domain progress from the brief-only program (i.e., *iter 0*) to the source harnesses selected on the in-distribution split.

Table 3 summarizes the primitives recovered from each harness. Note that retrieval programs are roughly twice the size of reasoning ones (≈ 26 vs. 14), carrying extra machinery to index and ground a document pool. More importantly, every skeleton step is functional; that is, the source-specific step count is 0.0 across all four domains, confirming that the logical plans cleanly transfer and source-specific details are confined to the decision logic on top.

3.2 Declarative Fidelity

Round-trip compilation tests whether the declarative description, which drops the original source code and retains only CREDO’s typed primitives (skeleton steps, beliefs, policies, prompt templates), can correctly reproduce the behavior of the original source harness. Choices such as evidence filtering, prompt order, fallback conditions, and sample count can affect behavior even when two programs share the same high-level strategy. Furthermore, the compiler can

Table 3: Catalog composition by transfer scope. Cells are mean ($N=3$) counts per program, as step/belief/policy/prompt template.

Domain	L1 (functional)	L2 (source-specific)	Total
Finance	5.0/6.7/7.3/1.0	0.0/2.0/5.3/0.0	27.3
Legal	3.3/7.6/7.7/0.0	0.0/3.7/2.7/1.0	26.0
Math	3.0/4.3/5.3/1.7	0.0/0.0/0.0/0.0	14.4
Polyreason	2.3/3.3/4.3/2.0	0.0/1.0/1.3/0.0	14.3

draw on its own model knowledge, so matching accuracy by itself would not distinguish faithful reconstruction from an entirely different harness. We therefore attempt to recover each original source harness by extracting CREDO primitives and directly compiling them back, running both harnesses on the same 100-question in-distribution split. An accurate recreation of the original harness should match both accuracy and cost, suggesting that our compiled harness faithfully matches the original searched harness rather than relying on the model to solve the task from its general priors.

We run this round-trip (RT) test on the three source harnesses per domain from §3.1. We recover each source with the abstractor, compile it back with the compiler, and evaluate both on the same 100-question in-distribution split, reporting accuracy, per-question cost, and the mean round-trip change $\Delta = \text{RT} - \text{orig}$. We also record the one-time cost and latency of a single recover-and-compile call against the search that produced the original harness.

Table 2 shows that CREDO’s RT preserves harness behavior across all four domains. We observe no major deviation in either accuracy, with at most a ± 0.011 difference (finance $+0.011$, legal -0.007 , math -0.010 , polyreason $+0.007$), or per-question cost (mean Δ within ± 0.013 ϵ/q). One RT costs about \$2.3–2.8 and takes only a few minutes, compared to \$43–72 and hours for the original Meta-Harness search.

3.3 Primitive Reuse

To determine whether our primitives generalize and can be effectively reused, we compiled harnesses for unseen content domains using functional primitives extracted in the same task family. We test four source→target transfers covering retrieval (finance↔legal) and reasoning (math↔polyreason). Every version receives the target brief (i.e., the dataset description and answer interface) and uses the same model pair, cross-distribution split, and judge. For the compiler, *brief-only* gets no catalog entries, *single-domain* receives the L1 primitives from one specified source, and *open-library* can access the full catalog containing every domain except the target. The results are shown in Fig. 3, with the target’s own Meta-Harness harness serving as the reference point.

For retrieval, CREDO’s primitives improve both target domains. Legal→finance (Fig. 3a) increases accuracy from 15.7 to 28.3, which falls within the run-to-run variation of the searched harness at 27.7, and finance→legal (Fig. 3b) increases accuracy from 43.4 to 66.3, recovering 76% of the distance to the searched result of 73.4. The open-library compiler produces similar values (28.2 vs. 28.3 for finance and 64.6 vs. 66.3 for legal) even though it must choose among primitives from all domains. In every replicate, the compiler selects the retrieval and model-routing primitives, rejects those associated with reasoning, and specializes the legal harness through appropriate slots (e.g., contract-attorney persona, clause / yes-no / no-clause output format) without modifying the templates.

Transfer for reasoning depends heavily on the operating point encoded by the source policies. Applying math primitives to polyreason (Fig. 3d) preserves a low inference cost of \$0.03–0.04 per question (5–7× below brief-only), yet the two compiled variants reach accuracies of 38.7 and 34.9, below the brief-only value of 43.3. In the opposite direction (Fig. 3c), the polyreason harness supplies an ensemble whose adjudication policy uses the format-independent answers_agree belief, improving math accuracy from 39.6 to 44.6 and 45.4. Thus, the compiler successfully transfers the recovered structure in both cases, but the selected beliefs and policies determine whether that structure occupies a useful region of the target cost-accuracy frontier, which is something that a future cost-based optimizer might allow the user to directly trade off (see §4).

Lastly, we note that the searched harnesses tend to overfit to the task domain compared to those compiled by CREDO. For example, the searched polyreason harness falls from 72% in-distribution to 47% on the cross-distribution split, and the searched math harness falls from 67% to 37%, below the corresponding baseline. CREDO’s compiled harnesses do not exhibit these same accuracy drops, even beating the searched program by roughly eight points on polyreason→math. Our approach also wins on cost, where one recover-and-compile pass costs about \$2.5 and takes two minutes compared to tens of dollars and hours for per-domain search.

4 Open Challenges

Our preliminary results demonstrate that CREDO can effectively recover and reuse declarative primitives from harnesses, but they also expose several open challenges that must be addressed to build a full-fledged system for managing agentic execution. In the following, we highlight some of these research opportunities.

Cost-Based Compilation. The current compiler selects primitives using role matching and natural-language metadata, making it a rule-based rewriter rather than a cost-based optimizer. Since each evaluated harness provides various measurements (e.g., accuracy, per-question inference cost, latency, belief selectivity), a cost-based compiler could use these statistics to enumerate compatible primitive combinations and maximize expected accuracy subject to a cost or latency budget, similar to a traditional query optimizer. The central challenges are estimating conditional operator quality, accounting for interactions among primitives, and remaining robust when sparse observations produce inaccurate estimates.

Declarative Catalog Maintenance. A catalog that only grows will eventually degrade by accumulating primitives that are either redundant, stale, erroneous, or dominated by some better version. Following physical design advisors [3], CREDO could evict primitives whose accuracy delta falls below a threshold, consolidate similar primitives into generalized specifications with wider slots, and flag primitives whose belief distributions have drifted. Selectivity drift is a natural staleness signal. For example, if `is_semantic_query` fired 70% of the time on a retrieval workload and now fires only 20%, the policies that depend on it need re-tuning.

Execution Context Recovery. CREDO currently recovers the harness program, but execution also depends on other context (e.g., system prompts, skill definitions, tool configurations) that may also result from search. As an example, the remaining finance→legal accuracy gap from §3.3 may reflect context that CREDO’s current abstractor does not capture. Extending recovery to these artifacts would allow the same transfer-scope and provenance machinery to distinguish potentially functional context (e.g., a retrieval persona or chunking strategy) from task-specific settings.

Pluggable Backends. The catalog stores substrate-independent optimization knowledge, so a recovered description could target execution systems other than Python harness code similar to how a logical query plan is decoupled from physical operator implementations. A LOTUS [23] backend, for example, could lower recovered steps into semantic operators with per-operator accuracy guarantees. Similarly, a backend that produces formal specifications could expose the assumptions required to close the gap between CREDO’s current behavioral fidelity tests and the semantic-equivalence guarantees available in the traditional UDF setting [8, 25]. Both directions require a precise contract for probabilistic or LLM-defined operators whose behavior can change across models and prompts.

5 Related Work

CREDO lies between systems that construct LLM workflows and database techniques that recover and optimize declarative structure. We compare these areas by the artifact available after construction, specifically whether it exposes the computation, runtime information, and resulting decisions independently of the original program.

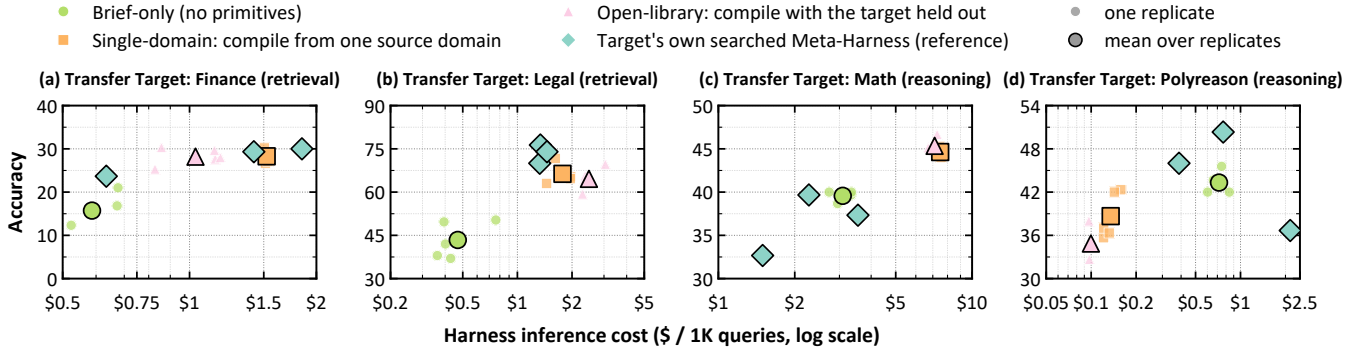


Figure 3: Primitive reuse, one plot per transfer target. Accuracy on the target’s cross-distribution split against harness inference cost.

Most orchestration frameworks retain the harness in the form in which a developer authored it. LangChain [2] and AutoGen [40], for example, represent control flow through program structure, prompt text, and inter-agent conversation, whereas agentic prompting methods such as ReAct [41] delegate more of the control flow to model reasoning. Declarative LLM data systems provide operators for specifying a computation and may optimize aspects of its execution [14, 19, 23, 29]. The recovery problem studied in this work begins after another process has already constructed the workflow.

Coding agents can search over complete harness programs by examining the source, evaluation results, and traces produced by earlier candidates [16], applying the evolutionary program search that coding agents use for algorithms and code more broadly [22, 30]. Prompt optimizers similarly search a narrower space and retain an optimized textual artifact for a fixed program [1], as do context optimizers that accumulate structured playbooks alongside it [45]. Program-level search can discover changes to operator order, conditional execution, model selection, and prompting in a single run, but its output combines these decisions in one implementation. CREDO accepts these results as input and attempts to recover reusable primitives at a granularity that permits separate cataloguing and transfer.

As mentioned, procedural UDF recovery supplies a database precedent for this separation. Systems such as Froid expose expressions and dependencies from imperative functions so that an optimizer can reason across the former procedural boundary [8, 25], and related work synthesizes queries from imperative database-backed application code [5]. CREDO adopts the representation-level objective, although its operators may invoke stochastic models and therefore lack a comparable algebra of semantics-preserving rewrites. Automatic tuning provides a complementary precedent in which physical design advisors choose access structures from workload cost estimates [3], potentially even learning from prior measurements to select DBMS settings [33, 44]; more recent work applies LLM-driven optimization to systems design beyond the database itself [10]. Harness compilation introduces different quantities to estimate, including output quality, model inference cost, and the selectivity of derived runtime attributes.

Some prior work attempts to reuse intermediates produced by agentic search. Existing approaches include structural priors over

typed operators [7], latent capability bases [35], code-level abstraction operators [46], and repositories of reusable workflows [38, 42]. Skill-library methods distill programs into reusable functions [32] or accumulate them from successful trajectories [34]. These artifacts range from individual capabilities to complete programs, whereas CREDO focuses on the intermediate plan representation needed to manage transfer as a systems decision.

6 Conclusion

This paper made the case for treating searched harnesses as managed declarative artifacts rather than retaining only their imperative implementations. CREDO recovers a typed description of a harness, records its primitives and provenance in a catalog, and compiles compatible structure into harnesses for new domains without another search. The preliminary evaluation shows that round-trip compilation closely preserves measured accuracy and cost while also demonstrating effective transferability across domains.

Beyond harness search, agentic workload execution also produces significant optimization knowledge that is currently discarded. The research agenda we described lays out a path toward making this information persistent and reusable, and CREDO is a first step in that direction. These are database systems problems (e.g., cost-based optimization, declarative specification of primitives), and the database community is well-positioned to solve them.

References

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*. arXiv:2507.19457 [cs.CL] <https://arxiv.org/abs/2507.19457>
- [2] Harrison Chase. 2022. LangChain. <https://github.com/langchain-ai/langchain>. Open-source framework for building LLM applications.
- [3] Surajit Chaudhuri and Vivek R. Narasayya. 1997. An Efficient Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, Athens, Greece, 146–155. <https://dl.acm.org/doi/10.5555/645923.673646>
- [4] Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A Dataset of Numerical Reasoning over Financial Data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 3697–3711.

- [5] Alvin Cheung, Armando Solar-Lezama, and Samuel Madden. 2013. Optimizing Database-Backed Applications with Query Synthesis. In *PLDI*. 3–14.
- [6] Jasper Dekoninck, Tim Gehringer, and Martin Vechev. 2026. ArXivMath: Evaluating LLMs on Mathematical Research Problems From Recent arXiv Papers. MathArena project, SRI Lab (ETH Zurich) and INSAIT. Benchmark release. <https://matharena.ai/arxivmath/>.
- [7] Shiyi Du, Jiayuan Liu, Weihua Du, Yue Huang, Jiayi Li, Yingtao Luo, Xiangliang Zhang, Vincent Conitzer, and Carl Kingsford. 2026. Why Search When You Can Transfer? Amortized Agentic Workflow Design from Structural Priors. arXiv:2604.25012 [cs.LG] <https://arxiv.org/abs/2604.25012>
- [8] Christian Duta, Denis Hirn, and Torsten Grust. 2020. Compiling PL/SQL Away. In *10th Conference on Innovative Data Systems Research (CIDR)*. www.cidrdb.org, Amsterdam, The Netherlands. <https://www.cidrdb.org/cidr2020/papers/p1-duta-cidr20.pdf>
- [9] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Qian, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. 2024. Omni-MATH: A Universal Olympiad Level Mathematical Benchmark for Large Language Models. (2024). arXiv:2410.07985 [cs.CL]
- [10] Pouya Hamadani, Pantea Karimi, Arash Nasr-Esfahany, Kimia Noorbakhsh, Joseph Chandler, Ali ParandehGheibi, Mohammad Alizadeh, and Hari Balakrishnan. 2025. Glia: A Human-Inspired AI for Automated Systems Design and Optimization. arXiv:2510.27176 [cs.AI]
- [11] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. 2024. OlympiadBench: A Challenging Benchmark for Promoting AGI with Olympiad-Level Bilingual Multimodal Scientific Problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 3828–3850.
- [12] Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021. CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review. In *NeurIPS Datasets and Benchmarks Track*.
- [13] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. 2023. FinanceBench: A New Benchmark for Financial Question Answering. (2023). arXiv:2311.11944 [cs.CL]
- [14] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations (ICLR)*.
- [15] Yuta Koreeda and Christopher D. Manning. 2021. ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts. In *Findings of the ACL: EMNLP 2021*. 1907–1919. doi:10.18653/v1/2021.findings-emnlp.164
- [16] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-Harness: End-to-End Optimization of Model Harnesses. arXiv preprint arXiv:2603.28052 (2026). Reference implementation: <https://github.com/stanford-iris-lab/meta-harness>.
- [17] Moxin Li, Fuli Feng, Hanwang Zhang, Xiangnan He, Fengbin Zhu, and Tat-Seng Chua. 2022. Learning to Imagine: Integrating Counterfactual Thinking in Neural Discrete Reasoning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Dublin, Ireland, 57–69.
- [18] Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. 2025. ZebraLogic: On the Scaling Limits of LLMs for Logical Reasoning. In *International Conference on Machine Learning (ICML)*.
- [19] Chunwei Liu, Matthew Russo, Michael J. Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael J. Franklin, Tim Kraska, Samuel Madden, Rana Shahout, and Gerardo Vitagliano. 2025. Palimpsest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *Conference on Innovative Data Systems Research (CIDR)*.
- [20] Duo Lu, Andrew Crotty, and Uğur Çetintemel. 2026. Credo: Declarative Control of LLM Pipelines via Beliefs and Policies. CoRR abs/2604.14401 (2026). <https://arxiv.org/abs/2604.14401>
- [21] Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. 2025. Towards Robust Mathematical Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 35418–35442. Introduces IMO-AnswerBench and IMO-ProofBench. arXiv:2511.01846 [cs.CL]
- [22] Alexander Novikov, Ngan Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislov Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131 [cs.AI] <https://arxiv.org/abs/2506.13131>
- [23] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (July 2025), 4171–4184. doi:10.14778/3749646.3749685
- [24] Nicholas Pipitone and Ghita Hour Alami. 2024. LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain. (2024). arXiv:2408.10343 [cs.AI]
- [25] Karthik Ramachandra, Kwanghyun Park, K. Venkatesh Emani, Alan Halverson, César A. Galindo-Legaria, and Conor Cunningham. 2017. Froid: Optimization of Imperative Programs in a Relational Database. *Proceedings of the VLDB Endowment* 11, 4 (2017), 432–444. doi:10.1145/3164135.3164140
- [26] Abhilasha Ravichander, Alan W. Black, Shomir Wilson, Thomas Norton, and Norman Sadeh. 2019. Question Answering for Privacy Policies: Combining Computational and Legal Perspectives. In *EMNLP-IJCNLP*. 4947–4958.
- [27] Varshini Reddy, Rik Koncel-Kedziorski, Viet Dac Lai, Michael Krumlick, Charles Lovering, and Chris Tanner. 2024. DocFinQA: A Long-Context Financial Reasoning Dataset. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 445–458.
- [28] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. In *First Conference on Language Modeling (COLM)*.
- [29] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proceedings of the VLDB Endowment* 18, 9 (May 2025), 3035–3048. doi:10.14778/3746405.3746426
- [30] Asankhaya Sharma. 2025. OpenEvolve: an open-source evolutionary coding agent. <https://github.com/algorithmicsuperintelligence/openevolve>
- [31] Zayne Sprague, Xi Ye, Kaj Bostrom, Swarat Chaudhuri, and Greg Durrett. 2024. MuSR: Testing the Limits of Chain-of-Thought with Multistep Soft Reasoning. In *International Conference on Learning Representations (ICLR)*.
- [32] Elias Stengel-Eskin, Archiki Prasad, and Mohit Bansal. 2024. ReGAL: Refactoring Programs to Discover Generalizable Abstractions. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 46605–46624.
- [33] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (Chicago, Illinois, USA) (SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 1009–1024. doi:10.1145/3035918.3064029
- [34] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291 [cs.AI]
- [35] Jialiang Wang, Shengxiang Xu, Hanmo Liu, Jiachuan Wang, Yuyu Luo, Shimin Di, Min-Ling Zhang, and Lei Chen. 2026. Learning to Compose for Cross-domain Agentic Workflow Generation. arXiv:2602.11114 [cs.AI] <https://arxiv.org/abs/2602.11114>
- [36] Steven H. Wang, Antoine Scardigli, Leonard Tang, Wei Chen, Dmitry Levkin, Anya Chen, Spencer Ball, Thomas Woodside, Oliver Zhang, and Dan Hendrycks. 2023. MAUD: An Expert-Annotated Legal NLP Dataset for Merger Agreement Understanding. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 16369–16382. arXiv:2301.00876 [cs.CL] doi:10.18653/v1/2023.emnlp-main.1019
- [37] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhira Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- [38] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 267)*. 63897–63911. arXiv:2409.07429 [cs.CL] <https://arxiv.org/abs/2409.07429>
- [39] Lindsey Linxi Wei, Shreya Shankar, Sepanta Zeighami, Yeounoh Chung, Fatma Ozcan, and Aditya G. Parameswaran. 2025. Multi-Objective Agentic Rewrites for Unstructured Data Processing. arXiv:2512.02289 [cs.DB] <https://arxiv.org/abs/2512.02289>
- [40] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI]
- [41] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2210.03629 [cs.CL] <https://arxiv.org/abs/2210.03629>

- [42] Lingzhi Yuan, Chenghao Deng, Fangxu Yu, Souradip Chakraborty, Mohammad Rostami, and Furong Huang. 2026. FlowBank: Query-Adaptive Agentic Workflows Optimization through Precompute-and-Reuse. arXiv:2606.11290 [cs.LG] <https://arxiv.org/abs/2606.11290>
- [43] Sepanta Zeighami, Shreya Shankar, and Aditya Parameswaran. 2025. Cut Costs, Not Accuracy: LLM-Powered Data Processing with Guarantees. arXiv:2509.02896 [cs.DB]
- [44] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiashu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, Minwei Ran, and Zekang Li. 2019. An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. In *SIGMOD*. 415–432.
- [45] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. In *ICLR*. arXiv:2510.04618 [cs.LG]
- [46] Mingming Zhao, Xiaokang Wei, Yuanqi Shao, Kaiwen Zhou, Lin Yang, Siwei Rao, Junhui Zhan, and Zhitang Chen. 2026. A²Flow: Automating Agentic Workflow Generation via Self-Adaptive Abstraction Operators. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. arXiv:2511.20693.
- [47] Fengbin Zhu, Wenqiang Lei, Fuli Feng, Chao Wang, Haozhou Zhang, and Tat-Seng Chua. 2022. Towards Complex Document Understanding By Discrete Reasoning. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*. Association for Computing Machinery, 4857–4866. doi:10.1145/3503161.3548422